

AIM

To implement file operations and access control in Linux.

SOFTWARE REQUIRED

- A Linux operating system
- GCC (GNU Compiler Collection) for compiling C programs
- Terminal or command line interface

THEORY

In Linux, every file and directory is associated with an owner, a group, and a set of permissions that determine who can read, write, or execute it. File operations such as creating, reading, writing, and closing files are performed through system calls that allow user-space programs to interact directly with the kernel.

Access control in Linux is enforced through permission bits associated with three categories of users: the owner (user), the group, and others. Each category can be granted read (r), write (w), and execute (x) permissions, which are commonly represented in a 10-character string such as -rw-r--r--, where the first character indicates the file type and the remaining nine characters represent the owner, group, and other permissions respectively.

The key system calls and functions used for file operations and access control are:

- **open()**: Opens or creates a file and returns a file descriptor.
- **read()**: Reads data from a file descriptor into a buffer.
- **write()**: Writes data from a buffer to a file descriptor.
- **close()**: Closes an open file descriptor.
- **stat()**: Retrieves file metadata, including permission bits, stored in the file's inode.
- **chmod()**: Changes the access permissions of a file.

By combining these system calls, a program can create a file, perform read and write operations on it, inspect its current permissions, and then modify those permissions to restrict or grant access, thereby demonstrating how access control is implemented at the operating system level.

PROCEDURE

1. Create a C Program:

Write a C program that performs file operations using `open()`, `read()`, `write()`, and `close()`, and implements access control using `stat()` and `chmod()`.

```
3.  #include <stdio.h>
4.  #include <fcntl.h>
5.  #include <unistd.h>
6.  #include <sys/stat.h>

7.  int main() {
8.      int fd;
9.      char text[] = "Hello Linux!\n";
10.     char buf[50];
11.
12.     // Create and write to file
13.     fd = open("test.txt", O_CREAT | O_WRONLY, 0644);
14.     write(fd, text, sizeof(text));
15.     close(fd);

16.     // Read file
17.     fd = open("test.txt", O_RDONLY);
18.     read(fd, buf, sizeof(buf));
19.     printf("File content: %s", buf);
20.     close(fd);
21.
22.     // Change file permissions
23.     chmod("test.txt", 0600);
24.
25.     printf("\nFile permissions changed to 600.\n");
26.
27.     return 0;
```

OUTPUT

```
Enter number of processes and resources: 5 3
```

```
Enter Allocation Matrix:
```

```
0  1  0
2  0  0
3  0  2
2  1  1
0  0  2
```

```
Enter Maximum Matrix:
```

```
7  5  3
3  2  2
9  0  2
2  2  2
4  3  3
```

```
Enter Available Resources:
```

```
3  3  2
```

```
Safe State
```

```
Safe Sequence: P1 P3 P4 P0 P2
```

CONCLUSION

This experiment successfully demonstrated file operations and access control in Linux. By using the `open()`, `read()`, `write()`, and `close()` system calls, we were able to create, read, and modify a file, while `stat()` and `chmod()` allowed us to inspect and enforce permission-based access control. The output confirmed that restricting a file's permissions to owner-read-only successfully prevented further write access, illustrating how the Linux kernel enforces access control at the file-system level.